Stash — Aggregator Scenario Report

Canonical engine at `web/src/lib/stashEngine.js` · Web direct import · Mobile via `mobile/lib/services/stash_engine.dart`.

All 74 assertions passed across 20 scenarios — web + mobile aggregators agree.

SCENARIO	AGGREGATED ITEMS	ASSERTIONS
Empty user — no receipt_items at all `empty_stash` New user, no purchases logged. Output is the empty list.	*(empty)*	✓ expect_count === 0
Single item from a single receipt `single_item_single_receipt` One receipt with one line item. One stash entry, qty 1, totalSpent = price.	• `milk gallon` · qty 1 · $4.99	✓ expect_count === 1 ✓ key="milk gallon" present ✓ milk gallon.qty === 1 ✓ milk gallon.totalSpent === 4.99 ✓ milk gallon.timesBought === 1 ✓ milk gallon.lastDate === "2026-05-30" ✓ milk gallon.category === "grocery" ✓ milk gallon.ratingCount === 0
Two receipts of the same item merge into one stash entry `two_receipts_same_item_merged` Same item_name bought twice on different days. One row, qty=2, totalSpent summed, lastDate = the later date.	• `bananas` · qty 5 · $3.95	✓ expect_count === 1 ✓ key="bananas" present ✓ bananas.qty === 5 ✓ bananas.totalSpent === 3.95 ✓ bananas.timesBought === 2 ✓ bananas.lastDate === "2026-05-25" ✓ bananas.lastReceiptId === "r2"

SCENARIO	AGGREGATED ITEMS	ASSERTIONS
Case-insensitive merge — "MILK" and "Milk" share a key `case_insensitive_merge` Different casing of the same product. Engine lowercases the key so both rows merge.	• `milk` · qty 2 · $10.00	• ✓ expect_count === 1 • ✓ key="milk" present • ✓ milk.qty === 2 • ✓ milk.totalSpent === 10
SKU-based merge — different names same SKU collapse `sku_preferred_over_name_for_merge` "Coca-Cola 12oz" and "Coke 12oz" share a SKU and merge into one stash item under the SKU key.	• `049000000443` · qty 2 · $3.78 · 2 stores	• ✓ expect_count === 1 • ✓ key="049000000443" present • ✓ 049000000443.qty === 2 • ✓ 049000000443.totalSpent === 3.78 • ✓ 049000000443.storeCount === 2
Returned items don't enter the stash `returned_items_excluded` A returned receipt_item (returned=true) is filtered before aggregation. Only non-returned rows remain.	• `headphones` · qty 1 · $200.00	• ✓ expect_count === 1 • ✓ key="headphones" present • ✓ headphones.qty === 1
Statement-import rows excluded (from_statement=true) `statement_imports_excluded` Receipt rows that came from a credit-card statement import aren't real product purchases. Engine filters them out of the stash.	• `eggs` · qty 1 · $5.00	• ✓ expect_count === 1 • ✓ key="eggs" present
Rows with blank item_name are excluded `empty_item_name_excluded` Corrupt rows (no name) can't render in the stash. Engine drops them silently.	• `apples` · qty 2 · $2.00	• ✓ expect_count === 1 • ✓ key="apples" present • ✓ apples.qty === 2

SCENARIO	AGGREGATED ITEMS	ASSERTIONS
Latest receipt's date / store / category wins `latest_receipt_metadata_wins` Three buys across stores. lastDate = most-recent date; lastStore = that store; category = most-recent category.	• `bread` · qty 3 · $9.00 · 3 stores	• ✓ expect_count === 1 • ✓ key="bread" present • ✓ bread.qty === 3 • ✓ bread.lastDate === "2026-05-30" • ✓ bread.lastReceiptId === "r3" • ✓ bread.lastStore === "Whole Foods" • ✓ bread.category === "bakery"
Same item across stores — distinct store count is right `multi_store_distinct_count` Same SKU bought at 3 different stores. storeCount = 3, stores list contains all three.	• `x123` · qty 3 · $39.00 · 3 stores	• ✓ expect_count === 1 • ✓ key="x123" present • ✓ x123.storeCount === 3
Ratings — max + avg computed across all rated rows `rating_max_and_avg` Three buys of the same item rated 3, 5, 4. ratingMax=5, ratingAvg=4.0, ratingCount=3.	• `yogurt` · qty 3 · $15.00 · 4.0★	• ✓ expect_count === 1 • ✓ key="yogurt" present • ✓ yogurt.ratingMax === 5 • ✓ yogurt.ratingAvg === 4 • ✓ yogurt.ratingCount === 3
Unrated rows don't affect ratingAvg `rating_null_ignored` Two buys — one rated 5, one unrated. ratingCount=1, ratingAvg=5 (the null row is excluded from the average).	• `tea` · qty 2 · $6.00 · 5.0★	• ✓ expect_count === 1 • ✓ key="tea" present • ✓ tea.ratingCount === 1 • ✓ tea.ratingAvg === 5 • ✓ tea.ratingMax === 5
qty as string coerced to int `qty_strings_coerced` Supabase sometimes returns numeric columns as strings. Engine coerces qty and price defensively.	• `apples` · qty 3 · $3.75	• ✓ expect_count === 1 • ✓ key="apples" present • ✓ apples.qty === 3 • ✓ apples.totalSpent === 3.75

SCENARIO	AGGREGATED ITEMS	ASSERTIONS
Item-level category null → falls back to receipt-level category `category_falls_back_to_receipt_category` When a row has no item.category but its parent receipt is categorized, that category propagates.	• `coffee filters` · qty 1 · $5.00	• ✓ expect_count === 1 • ✓ key="coffee filters" present • ✓ coffee filters.category === "household"
10+ receipts of same item — qty and timesBought sum correctly `large_qty_aggregate` Heavy aggregation stress: 10 separate receipts each with 2 qty. qty=20, timesBought=10.	• `coffee beans` · qty 20 · $240.00	• ✓ expect_count === 1 • ✓ key="coffee beans" present • ✓ coffee beans.qty === 20 • ✓ coffee beans.timesBought === 10 • ✓ coffee beans.totalSpent === 240
Mixed product list — distinct items each get their own entry `mixed_items_distinct_entries` Receipt with 4 different products. Aggregator produces 4 separate stash items.	• `apples` · qty 1 · $3.00 • `bananas` · qty 1 · $1.00 • `carrots` · qty 1 · $2.00 • `dates` · qty 1 · $4.00	• ✓ expect_count === 4
Sort by 'recent' — newest lastDate first `sort_recent_orders_by_lastDate_desc` Three items, different lastDates. sortStash('recent') orders newest first.	• `old` · qty 1 · $1.00 • `mid` · qty 1 · $1.00 • `new` · qty 1 · $1.00	• ✓ expect_count === 3 • ✓ sort 'recent' order = [new, mid, old]
Sort by 'spent' — highest totalSpent first `sort_spent_orders_by_totalSpent_desc` Three items, different totals. sortStash('spent') orders most-expensive first.	• `cheap` · qty 1 · $5.00 • `mid` · qty 1 · $50.00 • `pricey` · qty 1 · $500.00	• ✓ expect_count === 3 • ✓ sort 'spent' order = [pricey, mid, cheap]
filterStash by category — only items in that category `filter_by_category` Filter to category='produce'. Only the produce item survives.	• `apples` · qty 1 · $3.00 • `detergent` · qty 1 · $8.00	• ✓ expect_count === 2 • ✓ filter category='produce' → 1

SCENARIO	AGGREGATED ITEMS	ASSERTIONS
filterStash by query — case-insensitive name + sku match `filter_by_query` Search 'tea' matches Green Tea AND Green TEA Bags (and a SKU containing 'tea'). Detergent doesn't match.	• `green tea` · qty 1 · $4.00 • `green tea bags` · qty 1 · $6.00 • `detergent` · qty 1 · $8.00	• ✓ expect_count === 3 • ✓ filter query='tea' → 2

Cross-platform parity asserted by `mobile/test/services/stash_scenarios_test.dart`, reading the same `test-fixtures/stash-scenarios.json`.